Lifecycle Portfolio Allocation via Reinforcement Learning

CME 241 – RL for Stochastic Control Problems in Finance

Patrick Flanagan Jack Zhang Jeffrey Xue

Winter 2026

Outline

1. Problem: Lifecycle Portfolio Allocation as an MDP
2. Phase 2: Exact DP Solution (Backward Induction)
3. The Curse of Dimensionality
4. Phase 3: Reinforcement Learning Approach
5. Four RL Agents: Linear Q, SARSA, DQN, PPO
6. Results and Comparison
7. Conclusions

The Lifecycle Allocation Problem

Setting: An investor with T years until retirement must decide each year:

- How much wealth to **consume** (spend) vs. save
- How to **allocate** savings across stocks, bonds, and cash

Stochastic elements:

- **Income** follows a Markov chain (can be low, medium, or high)
- **Market regime** follows a Markov chain (bear, normal, or bull)
- **Asset returns** are random, conditioned on the current regime

Why this matters:

- This is exactly the problem that target-date retirement funds try to solve
- Traditional approaches use fixed rules (e.g., "60/40 stocks/bonds")
- Can RL discover better, *state-dependent* strategies?

MDP Formulation

State: $s_t = (t,\ W_t,\ y_t,\ z_t)$

- t = time period (year)
- W_t = current wealth
- $y_t \in \{\text{low, med, high}\}$ = income state
- $z_t \in \{\text{bear, normal, bull}\}$ = market regime

Action: $a_t = (\alpha_t,\ \mathbf{w}_t)$

- α_t = consumption fraction (0%–20%)
- $\mathbf{w}_t$ = portfolio weights summing to 1

Reward: $r_t = \beta^t \cdot u(c_t)$
Utility is constant relative risk aversion.

Objective:

$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{T-1} \beta^t u(c_t) + \beta^T u(W_T)\right]$$

CRRA Utility ($\gamma = 2$):

$$u(c) = \frac{c^{1-\gamma}}{1-\gamma} = -\frac{1}{c}$$

$\Rightarrow$ Agent is **risk-averse**

Transition Dynamics

Wealth transition:

$$W_{t+1} = \underbrace{(W_t - c_t)}_{\text{savings}} \cdot \underbrace{(\mathbf{w}_t \cdot \mathbf{R}_t)}_{\text{portfolio return}} + \underbrace{\text{income}(y_{t+1})}_{\text{labor income}}$$

Income transitions (Markov chain):

$$P_y = \begin{pmatrix} 0.70 & 0.25 & 0.05 \\ 0.15 & 0.70 & 0.15 \\ 0.05 & 0.25 & 0.70 \end{pmatrix} \quad \text{Income values: \$3, \$10, \$25}$$

Regime transitions (Markov chain):

$$P_z = \begin{pmatrix} 0.50 & 0.40 & 0.10 \\ 0.20 & 0.60 & 0.20 \\ 0.10 & 0.40 & 0.50 \end{pmatrix} \quad \text{Regimes: Bear, Normal, Bull}$$

Asset returns: 3 scenarios per regime (bad/medium/good) with regime-dependent probabilities. E.g., in a bear market, stocks return 0.82–1.02×; in bull, 0.98–1.25×.

Phase 2: Exact Solution via Backward Induction

Bellman equation:

$$V_t(w, y, z) = \max_{(\alpha, \mathbf{w})} \left[u(\alpha w) + \beta \sum_{y', z', k} p_{y'|y}\, p_{z'|z}\, p_k^{(z)}\; V_{t+1}(w', y', z') \right]$$

Algorithm:

1. Set $V_T(w) = u(w)$ for all terminal states
2. For $t = T-1, T-2, \ldots, 0$: try all 165 actions per state, store best

Phase 2 problem size:

- 121 wealth $\times$ 2 income $\times$ 2 regime $=$ 484 states/period
- $\times$ 25 periods $\times$ 165 actions
- $=$ **2.0 million** DP cells
- Solved in **0.4 seconds**

Key findings:

- DP-optimal beats all baselines on utility
- High stock allocation when young
- Shifts to bonds near retirement
- Consumption increases over time

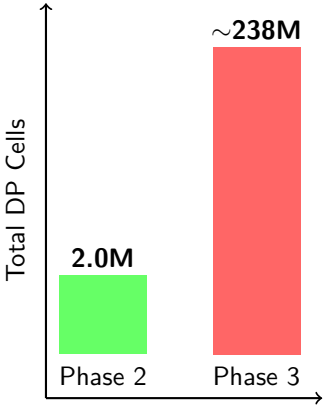
The Curse of Dimensionality

Making the problem *slightly* more realistic:

	Phase 2	**Phase 3**
Horizon	25	40
Wealth	121 grid pts	continuous
Income states	2	3
Regime states	2	3
Alloc. increments	25%	20%
Actions/state	165	210
DP cells (if discretized)†	**2.0M**	**∼238M**

† 500-pt wealth grid, 10% increments (hypothetical).

119× blowup – and still just 3 assets!



Solution: Function approx – learn $Q(s, a; \mathbf{w})$ from experience instead of tabulating.

$\Rightarrow$ **Reinforcement Learning**

Phase 3: RL Architecture

Environment (`PortfolioEnv`) – Gym-like interface:

- `reset()` → initial state (year 0, \$100 wealth, medium income, normal market)
- `step(action)` → (next_state, reward, done, info)
- State featurized as 8-dim vector: $\left[\underbrace{t/T}_{\text{time}}, \underbrace{\log W/\log W_{\max}}_{\text{wealth}}, \underbrace{\text{onehot}(y)}_{\text{3 bits}}, \underbrace{\text{onehot}(z)}_{\text{3 bits}}\right]$

Four RL agents:

	Linear Q	**SARSA**	**DQN**	**PPO**
Method	Q-learning	SARSA TD	Dueling DQN	Actor-Critic
Approx.	Linear (24-dim)	Linear (24-dim)	NN (256-256)	NN (256-256)
On/Off-policy	Off	**On**	Off	On
Parameters	5,040	5,040	∼122K	∼191K
Training	15K ep, 18s	15K ep, 15s	15K ep, 23 min	15K ep, 2 min

Linear Q, SARSA, DQN: $\epsilon = 1.0 \to 0.05$ over 8K episodes. PPO: $\epsilon = 0.3 \to 0.0$ over 5K episodes (primarily entropy-regularized).

Linear Q-Learning

Key idea: $Q(s, a; \mathbf{w}) = \mathbf{w}_a^\top \phi(s)$ – linear approximation with Q-learning updates ($210 \times 24 = 5{,}040$ params).

$$\phi(s) = [\underbrace{s_{1:8}}_{\text{8 raw}}, \underbrace{t^2, w^2, tw}_{\text{3 poly}}, \underbrace{t{\cdot}\text{oh}(y),\ t{\cdot}\text{oh}(z)}_{\text{6 t-crosses}}, \underbrace{w{\cdot}\text{oh}(y),\ w{\cdot}\text{oh}(z)}_{\text{6 w-crosses}}, \underbrace{1}_{\text{bias}}]$$

Off-policy TD update:

$$\Delta \mathbf{w}_a = \alpha \underbrace{\left(R + \gamma \cdot \max_{a'} Q(S', a'; \mathbf{w}) - Q(S, A; \mathbf{w})\right)}_{\text{TD error, clipped at } \pm 10} \cdot \phi(S)$$

Advantages:

- Tiny and fast ($\sim$18s to train)
- Interpretable weights per action
- Off-policy $\Rightarrow$ aggressive, sample-efficient learning

Limitations:

- Linear – misses complex nonlinear structure
- Deadly triad: FA + bootstrap + off-policy
- Requires hand-crafted features

Result: $\mathbb{E}[\text{Utility}] = -1.59$ (second best overall, behind DQN at -1.54).

SARSA: On-Policy TD Control

Same as Linear Q (features, params, ϵ-greedy). *One difference* in the TD target – use A' *actually taken*:

Linear Q (off-policy)

$$\boxed{\Delta \mathbf{w} = \alpha\big(R + \gamma \cdot \max_a Q(S', a) - Q(S, A)\big)\nabla Q}$$

"What's the *best* I *could* do?"

SARSA (on-policy)

$$\boxed{\Delta \mathbf{w} = \alpha\big(R + \gamma \cdot Q(S', A') - Q(S, A)\big)\nabla Q}$$

"What will I *actually* do?"

Why on-policy matters:

- Accounts for exploration cost – never overestimates Q-values from random actions
- **Safest** theoretical combination: linear FA + on-policy avoids the deadly triad
- Convergence to Q^* guaranteed (tabular, GLIE + Robbins-Monro); stable in practice
- Key detail: A' from update *must* be cached and reused as the next executed action

$\Rightarrow$ More conservative than Linear Q (-1.88 vs -1.59), but **safest policy** (Min $W_T = \$24.9$) and most theoretically grounded.

DQN: Deep Q-Network

Key idea: Replace linear $Q(s, a; \mathbf{w}) = \mathbf{w}_a^\top \phi(s)$ with a neural network that can capture *nonlinear* value functions.

Architecture (Dueling DQN): Shared trunk (8→256→256, Tanh) → Value stream (256→1) + Advantage stream (256→210). $Q = V + A - \bar{A}$. Total: ∼122K params.

PPO: Proximal Policy Optimization

Key idea: Instead of learning a value function and deriving the policy, *directly optimize the policy* $\pi_\theta(a|s)$.

Policy gradient theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) \cdot A^\pi(s,a)]$$

where $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is the **advantage**.

PPO clipped objective (prevents destructively large updates):

$$L(\theta) = \mathbb{E}\left[\min\left(\frac{\pi_\theta}{\pi_{\text{old}}}A,\ \ \text{clip}\left(\frac{\pi_\theta}{\pi_{\text{old}}}, 1-\epsilon, 1+\epsilon\right) A\right)\right]$$

Actor-Critic: Separate networks – Actor π_θ (policy, $\sim$122K params) + Critic V_ϕ (value/GAE, $\sim$68K). Total: $\sim$191K. Both: 256-256 Tanh, orthogonal init.

Training Convergence

Key observations:

- **PPO** converges fastest and to the best training return but doesn't stabilize well
- **Linear Q & SARSA** converge similarly; SARSA slightly slower (on-policy)
- **DQN** slower due to filling replay buffer

Training times:

Linear Q: ∼18s
SARSA: ∼15s
DQN: ∼23 min
PPO: ∼2 min

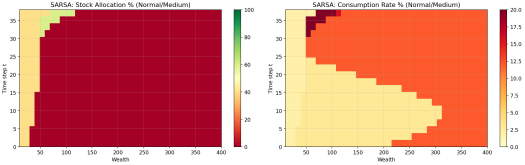
Monte Carlo Evaluation (2,000 Simulated Lifetimes)

Strategy	$\mathbb{E}$[Utility]	$\mathbb{E}[W_T]$	Med$[W_T]$	Std$[W_T]$	P_{10}	P_{90}
DP Optimal	$-$**1.46**	77	75	29	42	119
DQN	-1.54	115	107	52	58	186
Linear Q	-1.59	96	87	34	60	144
SARSA	-1.88	99	93	35	58	150
PPO	-1.95	91	81	43	45	150
Static 60/40	-2.63	487	470	166	281	747
Equal 1/3	-2.75	408	396	112	271	564
Glidepath 80$\rightarrow$20	-2.63	489	474	140	320	697

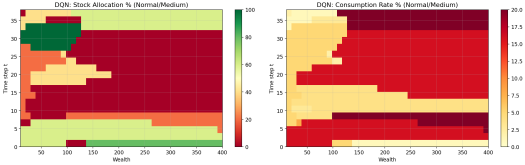
All four RL agents beat all three baselines on utility.

- RL agents have *lower* terminal wealth but *higher* utility
- **DQN** and **Linear Q** achieves the best expected utility, within 10% of **DP Optimal**
- **SARSA** (-1.88) and **PPO** (-1.95) – still far better than any baseline (-2.63)
- Baselines accumulate large wealth ($487+) but don't optimize consumption timing

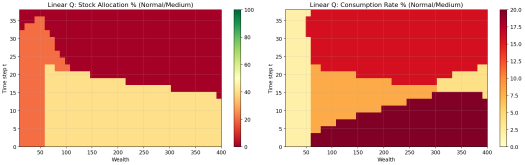
Learned Policies: What Do the Agents Do?



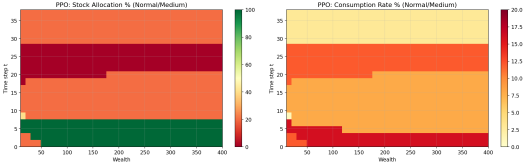
SARSA Policy



DQN Policy



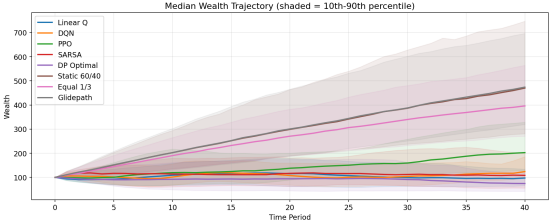
Linear Q Policy

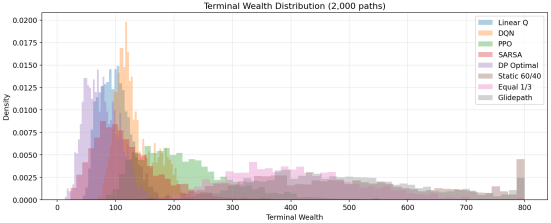


PPO Policy

Reading the heatmaps: y-axis = time (year), x-axis = wealth level. Color = stock allocation or consumption chosen by the agent. All agents learn intuitive patterns: higher stock allocation when young (time to recover from crashes), shift toward bonds/cash near retirement.

Wealth Trajectories and Terminal Wealth





Left: Sample wealth trajectories over 40 years. RL agents (consuming along the way) have lower wealth paths than baselines (which only invest).
Right: Terminal wealth distributions. Baselines produce higher but more spread-out terminal wealth; RL agents produce tighter distributions centered on lower values – by design.

Comparing All Four RL Agents

	Linear Q	**SARSA**	**DQN**	**PPO**
Update target	$\max_a Q(s', a)$	$Q(s', a')$	$\max_a Q(s', a)$	Policy gradient
Function approx	Linear (24-dim)	Linear (24-dim)	Dueling (256-256)	Sep. A-C (256-256)
Parameters	5,040	5,040	$\sim$122K	$\sim$191K
Training time	$\sim$18s	$\sim$15s	$\sim$23 min	$\sim$2 min
$\mathbb{E}$**[Utility]**	-1.59	-1.88	$\mathbf{-1.54}$	-1.95
C-Sharpe	18.30	10.25	**26.53**	12.58
Min W_T	**33.2**	24.9	18.3	20.5

Takeaways:

- **DQN wins on utility** (-1.54): Dueling arch + large replay buffer captures nonlinear value
- **Linear agents win on efficiency**: 5,040 params, trains in seconds, interpretable policy
- **Off-policy** $>$ **on-policy on utility**: Linear Q beats SARSA; DQN beats PPO
- **All RL agents crush all baselines**: optimizing the right objective matters most

Conclusions

1. **Exact DP works but doesn't scale.** Phase 2's backward induction gives the provably optimal solution in 0.4s, but a 119× blowup to Phase 3 parameters makes it infeasible.
2. **RL handles continuous wealth and richer state spaces.** Function approximation replaces the DP table, scaling linearly in episodes rather than exponentially in state dimensions.
3. **All four RL agents beat all baselines on utility.** Off-policy methods (DQN, Linear Q) achieve the best utility; on-policy (SARSA, PPO) are simpler and faster. SARSA's on-policy nature gives the most conservative policy.
4. **RL optimizes the right objective.** Static strategies implicitly optimize wealth. RL directly optimizes *lifetime utility*, discovering that a risk-averse investor should consume more and invest conservatively – matching financial theory (Merton's solution).

Future work: Continuous actions, more assets, transaction costs, real data calibration, and improve current models.

Thank You!

Questions?